



NOWITNESS

Security Audit Report

FluidTokens Smart Contracts Security Audit

Date	February 27, 2026
Client	FluidTokens
Project	FluidTokens Cardano Loans V3
Version	0.0.0

Contents

Disclosure	1
Disclaimer	2
Scope	4
Assessment Overview	5
Assessment Components	6
Executive Summary	7
Code Base	10
Severity Classification	12
Finding Severity Ratings	13
Findings	14
[C 501] Incorrect fold initialization	16
[C 502] Miscalculation of Interest Rate	18
[C 503] Currency Unit Mismatch 1	20
[C 504] Assets Mismatch in Repayment Validation	23
[C 505] Unhandled Partial Liquidation	24
[H 401] Unvalidated Negative Equity during Liquidation	26
[H 402] Incorrect Usage of maxAllowedSlippage	30
[H 403] Inadequate Slippage Protection	32
[H 404] Incorrect Calculation of Collateral Price	38
[H 405] Collateral Neglect in Compound Action	39
[M 301] Currency Unit Mismatch 2	42
[M 302] Inversion of Arguments in Slippage Check	46
[M 303] Unhandled Asset Conversion Failures	47
[M 304] Negative liquidationFeePerMille Exploit	50
[I 101] Non-idiomatic list traversal 1	53
[I 102] Non-idiomatic list traversal 2	56
[I 103] ADA policy literal	58
[I 104] Equity Currency Choice	59
[I 105] Double traversal during NFT mint validation	61
[I 106] Redundant equity branch in calculation	63
[I 107] Code Duplication of Minswap Types and Functions	65
[I 108] Duplicate Implementation of Minswap Functionality	68
[I 109] Code Optimization in lm_liquidate_and_convert_action.ak	70
[I 110] Redundant Calculation of Remaining Debt	72
[I 111] Lack of Unit Tests and Property-Based Tests	74

[I 112]	Documentation Typographical Errors and Inaccuracies	75
[I 113]	Configuration NFT Minting Output Conflict	77

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **FluidTokens** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

External oracle dependency

This protocol relies on third-party oracles for price feeds, which were not within the scope of this review. The security of these oracles is critical to the overall safety and functionality of the protocol but was not assessed in this engagement. Users and stakeholders should be aware of this dependency and consider the security implications of relying on external data sources.

External DEX dependency

This protocol relies on external DEXes for swapping assets, which were not within the scope of this review. The security of these DEXes is critical to the overall safety and functionality of the protocol but was not assessed in this engagement. Users and stakeholders should be aware of this dependency and consider the security implications of relying on third-party platforms for critical operations.

User-Provided Data Dependency

This protocol relies on the correctness of a specific datum, which is essential for its operation. The security of this datum is critical to the overall safety and functionality of the protocol. The validators rely on these data, and they assume the user has verified them. It is important for users to ensure the accuracy and integrity of this datum, as any discrepancies could lead to unintended consequences or vulnerabilities in the protocol's operation. Therefore, users and stakeholders should be aware of this dependency and take necessary precautions to verify the data they interact with when using the protocol.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **November 24, 2025** to **February 04, 2026**, **FluidTokens** engaged No Witness Labs to conduct a comprehensive security assessment of the **FluidTokens Cardano Loans V3** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

Protocol Overview

FluidTokens Lending V3 is a Cardano lending protocol implemented as a set of official smart contracts. The logic is intended to be highly customizable, such that the same contracts may be used for different flavours of lending.

At a high level, the protocol supports:

- Static and dynamic liquidity pools (dynamic pools establish borrowable amounts via LTV and oracles), with native multisig wallet support.
- Borrowing from multiple pools in the same transaction, and borrower-created loan requests that lenders can accept.
- Pools that can accept different collaterals, with each loan using exactly one collateral.
- Installments, an initial grace period, and penalty fees for late repayments.
- Four liquidation types (including no-liquidation, dutch auction, and oracle-based variants) and three repayment formulas (including amortization and perpetual loans).
- Unique NFTs identifying pools, requests, loans, and repayments, and bonds (NFTs) representing lender and borrower positions.
- Oracle feeds expressed as tokenX/lovelace with aggregated, pooled, and dedicated feed types, plus CIP-113 (smart tokens) support for principal, collateral, and oracle feeds.
- Optional automatic liquidations and optional automatic principal compounding.

The protocol is described as non-custodial (assets remain owned by users), with repayments sent to the lender-specified address with a specified datum, and is intended to be composable with other Cardano DeFi protocols.

Automatic liquidations and optional compounding

Lenders can enable automatic loan liquidations and optional automatic principal compounding to the pool. Bots compete to execute these transactions and earn liquidation and compounding fees that are set by the lender.

Pool setup requirements

When a lender creates a liquidity pool, in the same transaction they must also mint a PoolManager with the `poolId` field equal to the ID of the new pool:

- The pool `lenderAuth` field must be set as the Withdraw script hash of the PoolManager.
- The pool `lenderBondAddress` field must be set as the Spend script hash of the LenderManager.
- The pool `lenderBondInlineDatumHash` field must be the `blake2b_256` hash of the LenderManager datum.
- `shouldLiquidationConvertToPrincipal` must be set to `True` if bots should convert liquidated collateral into principal (either paying it in advance or converting it through a DEX); otherwise bots liquidate collateral and place it in the LenderManager spend script.
- For automatic compounding, `PoolManagerId` must be set as the NFT AssetName of the newly created PoolManager; otherwise it is left empty.

Operational routing

- When a new loan is created, the lender's bond is locked in the LenderManager spend script.
- Each borrower repayment (installment, full amount, recast) is sent to the LenderManager spend script.
- To cancel or edit a pool, the lender also burns the corresponding PoolManager.
- The lender can edit the PoolManager datum to change ownership of the lender bond or to change the compounding fee.

Liquidation conversion behavior

- If `shouldLiquidationConvertToPrincipal` is `False`, bots can only liquidate the loan's collateral, and the result is placed in the LenderManager spend script.
- If `shouldLiquidationConvertToPrincipal` is `True`, bots must liquidate and convert collateral to principal either by:
 - placing an order on a DEX (if the calculated slippage is within acceptable limits), or
 - anticipating the principal to the lender and keeping the collateral, then placing the result in the LenderManager spend script.

Compounding behavior

- If any UTxO in the LenderManager spend script contains principal and points to the proper PoolManager, bots can compound liquidity into the pool and take an additional fee.
- For loans that correctly point to a PoolManager, bots can liquidate, pay in advance, and compound in one transaction, taking both liquidation and compounding fees.

Manual lender capabilities

- The lender can manually withdraw assets that belong to them from the LenderManager spend script.
- The lender can withdraw their bonds from the LenderManager spend script.

Code Base

Repository

<https://github.com/FluidTokens/ft-cardano-loans-v3.git>

Branch

newLiquidation

Commit

9e038e7ef8ee3138f4042ad7ce5e1c8db87a10c1

Files Audited

Files	SHA256 Checksum
validators/loan/loan_claim_action.ak	ff9aee5080991aaa2ad52d1ef594d81fb3bbd2bb7ba0b4ba41c934f5d72d132c
validators/loan/loan_change_collateral_action.ak	507858467c6af3c296687514054a9665f796a23301f83322265b31c41540d3d4
validators/lender_manager.ak	bb9cb038fe968bcd29f10e099d40de9ed9e7c5f386c0bc760fd0b015667c11cc
validators/lender-manager/lm_compound_action.ak	43b17af8da8585783a8d5e6fd225b5139eb9b749179df21ed9d7e43435473262
validators/lender-manager/lm_config.ak	0011dba9f70cebfd58865867a6a143ca95fe5b352c7d22bde8cd5873cc2e6494
validators/lender-manager/lm_liquidate_action.ak	bc16fa820ca71f6af25c4baa914796a27026252926a4da34c7a64412e8a9ca09
validators/lender-manager/lm_liquidate_and_convert_action.ak	1d278f9f9701ac719743cc5e9209fa35c21cfae3bd5ab7d7e68a18cdac3f5805
validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak	cb97e068e0ef22112686c5c27594a363c0c6425ed7f47ee41c51c57ce97972db
validators/lender-manager/lm_liquidate_convert_and_compound_action.ak	414f6c6822a99100d2d92f6c7e0f2682fe2a68079ad272997bf7ba9a0309e5c6
validators/lender-manager/lm_liquidate_pay_in_advance_and_compound_action.ak	58bdc158505a9864d7e7e4a737ce7abf1f02001c272b4a2325967b969233d8f6

Files	SHA256 Checksum
validators/lender-manager/lm_withdraw_assets_action.ak	4776deca2cec7e6f4c245bc9273cf64c893e7bcc197c694a72f097861056be53
validators/lender-manager/lm_withdraw_bonds_action.ak	4ff25e233938899eb33edc488d87f64daaf1b1d48c4c40d77afe469dc6eb17ad

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	5
	4	High	5
	3	Medium	4
	2	Low	0
	1	Informational	13
Total Findings			27

Findings

ID	Title	Severity	Status
501	Incorrect fold initialization	Critical	Resolved
502	Miscalculation of Interest Rate	Critical	Resolved
503	Currency Unit Mismatch 1	Critical	Resolved
504	Assets Mismatch in Repayment Validation	Critical	Resolved
505	Unhandled Partial Liquidation	Critical	Resolved
401	Unvalidated Negative Equity during Liquidation	High	Resolved
402	Incorrect Usage of maxAllowedSlippage	High	Resolved
403	Inadequate Slippage Protection	High	Resolved
404	Incorrect Calculation of Collateral Price	High	Resolved
405	Collateral Neglect in Compound Action	High	Resolved
301	Currency Unit Mismatch 2	Medium	Resolved
302	Inversion of Arguments in Slippage Check	Medium	Resolved
303	Unhandled Asset Conversion Failures	Medium	Resolved
304	Negative <code>liquidationFeePerMille</code> Exploit	Medium	Resolved

ID	Title	Severity	Status
101	Non-idiomatic list traversal 1	Informational	Resolved
102	Non-idiomatic list traversal 2	Informational	Resolved
103	ADA policy literal	Informational	Resolved
104	Equity Currency Choice	Informational	Resolved
105	Double traversal during NFT mint validation	Informational	Resolved
106	Redundant equity branch in calculation	Informational	Resolved
107	Code Duplication of Minswap Types and Functions	Informational	Resolved
108	Duplicate Implementation of Minswap Functionality	Informational	Resolved
109	Code Optimization in <code>lm_liquidate_and_convert_action.ak</code>	Informational	Resolved
110	Redundant Calculation of Remaining Debt	Informational	Resolved
111	Lack of Unit Tests and Property-Based Tests	Informational	Acknowledged
112	Documentation Typographical Errors and Inaccuracies	Informational	Resolved
113	Configuration NFT Minting Output Conflict	Informational	Acknowledged

[C 501] Incorrect fold initialization

	Level	Severity	Status
	5	Critical	Resolved

Category: Implementation bug / Unintended failure

Affected Files:

- `validators/lender-manager/lm_liquidate_action.ak`
- `validators/lender-manager/lm_liquidate_and_convert_action.ak`
- `validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak`

Description

In several lender-manager validators, loan inputs are validated using `list.indexed_foldr` with a Boolean accumulator:

```

1  list.indexed_foldr(
2    loanInputs,
3    False,
4    fn(index, loanInput, result) {
5      // As all inputs must return True, if any of previous input returned False, then
5      fail
6      // The last input is not checked but will return its value at the end, so if False
6      withdraw will fail
7      expect result
8      ...
9    }
10 )

```

✳️ Aiken

The accumulator is incorrectly initialized to `False`. Which leads to the following issues:

- On the first iteration, `result` is `False`, so `expect result` will fail even before any per-input checks can influence it.
- Conceptually, the intended invariant “all previous inputs passed” requires an initial accumulator of `True`, not `False`.

Impact

- `lm_liquidate_action.ak`: All liquidation attempts will fail.
- `lm_liquidate_and_convert_action.ak`: All liquidation-and-convert attempts will fail.
- `lm_liquidate_and_pay_in_advance_action.ak`: All liquidation-and-pay-in-advance attempts will fail.
- Practically, these lender-manager actions are blocked, preventing automatic liquidations and exposing lenders to undercollateralized positions.

Recommendation

Choose one of the following options:

- **Option A:** Apply the recommended fix from Finding **F101** (*Non-idiomatic list traversal 1*):
 1. Create an `indexed_all` utility function in `lib/fluidtokens/utls.ak` that encapsulates the pattern of checking that all elements satisfy a predicate.
 2. Replace the existing `list.indexed_foldr` usages in the affected validators with calls to `utls.indexed_all`, passing the per-input check as the predicate.
- **Option B:** Initialize the Boolean accumulator to `True` so that:
 - The first `expect result` succeeds by default, and
 - Subsequent steps can flip the accumulator to `False` if any input fails.

Resolution

Resolved by applying the recommended fix from Finding **F101** (*Non-idiomatic list traversal 1*), which replaces the incorrectly initialized `list.indexed_foldr` pattern with a more appropriate `indexed_all` utility function.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[C 502] Miscalculation of Interest Rate

	Level	Severity	Status
	5	Critical	Resolved

Category: Calculation Error

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

In the `lm_liquidate_and_convert_action.ak` file, there is a miscalculation in the debt repayment logic. The code currently uses `rational.from_int(interestRate)` when calculating the remaining debt, which may not accurately represent the interest rate as a fraction. This could lead to incorrect calculations of the minimum amount that needs to be received during liquidation.

```

1  amountOut >= finance.get_remaining_debt(
2      repaymentMode,
3      rational.from_int(principalAmount),
4      rational.from_int(interestRate),
5      totalInstallments,
6      repaidInstallments,
7      installmentPeriod,
8      initialGracePeriod,
9      validTo - lendDate,
10 )

```

 Aiken

The interest rate should be represented as a rational number, using `rational.new(interestRate, 10000)`, to accurately reflect the percentage rate in the calculations.

Moreover, the use of `validTo - lendDate` instead of `validFrom - lendDate` may also contribute to an inflated debt calculation, as it extends the period considered for interest accrual.

Impact

Using `rational.from_int(interestRate)` instead of `rational.new(interestRate, 10000)` can lead to an inflated interest calculation. This results in a higher remaining debt calculation, making it more likely for the validator to fail the check during automatic liquidation. This could prevent legitimate liquidations from occurring, potentially leading to under-collateralized loans and financial losses for lenders.

Recommendation

1. Fix the calculation of the interest rate to ensure it accurately reflects the intended percentage before calculating the remaining debt.

2. Use `validFrom - lendDate` instead of `validTo - lendDate` to accurately calculate the time period for interest accrual.

```

1  amountOut >= finance.get_remaining_debt(
2      repaymentMode,
3      rational.from_int(principalAmount),
4      rational.new(interestRate, 10000),
5      totalInstallments,
6      repaidInstallments,
7      installmentPeriod,
8      initialGracePeriod,
9      validFrom - lendDate,
10 )

```

Aiken

*Note: If the recommendation from Finding **F110** is implemented, this issue is no longer relevant.*

Resolution

Resolved by implementing the recommendation from Finding **F110**, which recommends importing the remaining debt instead of recalculating it.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[C 503] Currency Unit Mismatch 1

	Level	Severity	Status
	5	Critical	Resolved

Category: Value accounting / Unit mismatch

Affected File: `validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak`

Description

During automated liquidation, the lender's collateral share is computed by subtracting `loanInputAction.equity` from `loanCollateralAmount` when `partialLiquidationPenaltyPerMille > 0`.

However, this logic implicitly assumes `loanInputAction.equity` is denominated in collateral currency, while `loanInputAction.equity` can be denominated in principal currency. Since `loanCollateralAmount` is always in collateral currency, subtracting equity expressed in principal currency is a unit mismatch and produces an incorrect result.

The relevant code snippet is:

```
1 let loanCollateralThatLenderShouldReceive =
2     if partialLiquidationPenaltyPerMille > 0 {
3         loanCollateralAmount - loanInputAction.equity - liquidationFee
4     } else {
5         loanCollateralAmount - liquidationFee
6     }
```

 Aiken

Impact

The outcome depends on the price ratio between collateral and principal currencies.

Let: $\text{ratio} = \text{collateral price} / \text{principal price}$

When $\text{ratio} > 1$,

- `equity_principal > equity_collateral`
- `actual_equity > expected_equity`
- `calculated_lender_collateral < expected_lender_collateral`
- Lender receives less collateral than they are entitled to.

When $\text{ratio} < 1$,

- `equity_principal < equity_collateral`
- `actual_equity < expected_equity`
- `calculated_lender_collateral > expected_lender_collateral`
- `convertedLoanCollateralToPrincipalAmount` is larger than intended, causing the lender to receive more principal than they are entitled to.

- Since this validator is a pay-in-advance action, the excess principal payout is withdrawn from the protocol's reserves, resulting in a loss to the protocol.
- `validators/loan/loan_claim_action.ak` 's validator enforces borrower to receive the right amount (No loss to the Borrower)

Where:

- `equity_principal` is the equity expressed in principal currency
- `equity_collateral` is the equity expressed in collateral currency
- `actual_equity` is the equity value used in the calculation (i.e., `loanInputAction.equity`)
- `expected_equity` is the correct equity value that should have been used
- `calculated_lender_collateral` is the lender collateral amount computed by the script (i.e., `loanCollateralThatLenderShouldReceive`)
- `expected_lender_collateral` is the correct lender collateral amount that should have been computed

Attack Scenario:

Scenario 1: Lender receives less collateral than entitled

1. Lender creates a pool with the following Conditions:
 - `equityInPrincipalCurrency = True` at `PoolDatum.commonData.LiquidationMode`
 - `partialLiquidationPenaltyPerMille > 0` at `PoolDatum.commonData.LiquidationMode`
 - Collateral currency is more expensive than principal currency (i.e., `ratio > 1`).
2. Borrower takes a loan from that pool.
3. Loan reaches liquidation conditions.
4. Borrower executes automated liquidation, using `lm_liquidate_and_pay_in_advance_action.ak` validator action.
5. Due to the currency mismatch, the script computes a smaller `loanCollateralThatLenderShouldReceive` than intended.
6. Lender receives less collateral than they are entitled to. And the rest of the collateral are returned to the borrower.

Note: Attacker here is the Borrower or whoever executes the liquidation.

Scenario 2: Lender receives more principal than entitled (Protocol Loss)

1. Lender creates a pool with the following Conditions:
 - `equityInPrincipalCurrency = True` at `PoolDatum.commonData.LiquidationMode`
 - `partialLiquidationPenaltyPerMille > 0` at `PoolDatum.commonData.LiquidationMode`
 - Collateral currency is less expensive than principal currency (i.e., `ratio < 1`).
2. Borrower takes a loan from that pool.
3. Loan reaches liquidation conditions.
4. Lender executes automated liquidation, using `lm_liquidate_and_pay_in_advance_action.ak` validator action.

5. Due to the currency mismatch, the script computes a larger `loanCollateralThatLenderShouldReceive` than intended.
6. Lender receives more principal than they are entitled to. The excess principal payout is withdrawn from the protocol's reserves, resulting in a loss to the protocol.

Note: Attacker here can play both roles of Lender and Borrower to exploit this vulnerability.

Recommendation

Ensure the equity subtracted from `loanCollateralAmount` is always in collateral currency.

This is achieved by adding the following check:

```
1 expect equityInPrincipalCurrency == false
```

 Aiken

Suggested patch:

```
1  ...
2  validator actionValidator(...) {
3    withdraw(...) {
4      ...
5      let liquidatedCollateralOutputsAreCorrect =
6        list.indexed_foldr(
7          loanInputs,
8          True,
9          fn(index, loanInput, result) {
10           ...
11           expect Liquidation {
12             partialLiquidationPenaltyPerMille,
13 +           equityInPrincipalCurrency,
14             ..
15           } = loanInputAction.liquidationMode
16 +           expect equityInPrincipalCurrency == False
17           ...
18         }
19       )
20     ...
21   }
22 }
```

 diff

Resolution

Resolved by adding validation to ensure `equityInPrincipalCurrency` is `False`

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[C 504] Assets Mismatch in Repayment Validation

	Level	Severity	Status
	5	Critical	Resolved

Category: Assets Mismatch

Affected File: `validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak`

Description

In the `lm_liquidate_and_pay_in_advance_action.ak` file, there is a critical vulnerability in the `validate_repayment_output` function call. The `repaymentAsset` parameter is incorrectly set to use collateral units, while the `repaymentAmount` parameter is specified in principal units. This mismatch can lead to incorrect validation of repayment outputs, potentially allowing malicious actors to exploit the system by providing insufficient repayments that are still considered valid.

Impact

This vulnerability can lead to significant financial losses for lenders, as it may allow borrowers to repay less than what is owed while still passing validation checks.

Depending on the collateral-to-principal conversion rates, the validator can either:

- Accept repayments that are lower than the actual principal owed, leading to underpayment.
- Reject valid repayments, causing unnecessary transaction failures. Which may lead to under-collateralized loans.

Recommendation

Replace the `repaymentAsset` parameter in the `validate_repayment_output` function call to use the principal asset instead of the collateral asset. The corrected line should be:

```
1 repaymentAsset: Asset(principalAsset.policyId, principalAsset.assetName), ak
```

This change ensures that both the `repaymentAsset` and `repaymentAmount` parameters are aligned in terms of units, thereby maintaining the integrity of the repayment validation process.

Resolution

Resolved by updating the `validate_repayment_output` function call to use the principal asset for the `repaymentAsset` parameter, ensuring consistency between `repaymentAsset` and `repaymentAmount` units.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[C 505] Unhandled Partial Liquidation

	Level	Severity	Status
	5	Critical	Resolved

Category: Value accounting

Affected File: `validators/lender-manager/lm_liquidate_pay_in_advance_and_compound_action.ak`

Description

In the `lm_liquidate_pay_in_advance_and_compound_action.ak` file, `loanCollateralToSend` is calculated by subtracting only the `liquidationFee` from `loanCollateralAmount`. While this is correct for full liquidations, it fails to account for the borrower's equity in cases of partial liquidation.

The relevant code snippet is:

```
1 let loanCollateralToSend = loanCollateralAmount - liquidationFee
2 let convertedLoanCollateralToPrincipalAmount =
3   finance.convertFromAToBWithOracles(
4     collateralOraclePriceFeed,
5     principalOraclePriceFeed,
6     rational.from_int(loanCollateralToSend),
7   )
```



The current implementation does not account for the borrower's equity in the loan when calculating the amount of collateral to be converted to principal. This oversight can lead to overpayment to lenders, resulting in financial losses for the protocol.

Impact

This vulnerability can lead to financial losses for the protocol, as it may result in overpayment to lenders during the liquidation process. By not accounting for the borrower's equity, the protocol risks depleting its reserves, which could impact its ability to operate effectively and maintain liquidity.

Recommendation

1. Adjust the calculation of `loanCollateralToSend` to account for the borrower's equity in the loan. The corrected line should be:

```
1 let loanCollateralToSend = loanCollateralAmount - loanInputAction.equity -
  liquidationFee
```



2. Ensure that `loanInputAction.equity` is expressed in collateral units before performing the subtraction:

```
1 expect Liquidation { equityInPrincipalCurrency, .. } =
2   loanInputAction.liquidationMode
```



```
3 expect equityInPrincipalCurrency == false
```

Resolution

Resolved by adjusting the calculation of `loanCollateralToSend` to account for the borrower's equity in partial liquidations and ensuring `loanInputAction.equity` is expressed in collateral units.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[H 401] Unvalidated Negative Equity during Liquidation

	Level	Severity	Status
	4	High	Resolved

Category: Cross-validator invariants / Negative values / Optimization

Affected File: `validators/loan/loan_claim_action.ak`

Description

The liquidation path in `loan_claim_action.ak` does not enforce basic invariants on `inputAction.equity`, even though this value is exported and reused by other validators (e.g. `validators/lender-manager/lm_liquidate_action.ak`), which assume it is correctly computed and non-negative.

- `inputAction.equity` is only checked for equality against `get_equity(...)` / `get_equity_in_collateral_currency(...)`, but these helpers can return negative values. The validator does not forbid negative `inputAction.equity`.
- When `partialLiquidationPenaltyPerMille < 0`, the validator still computes a (boosted) equity amount, but the borrower-compensation `or { ... }` block (the third `or` in the snippet below) allows the transaction to:
 - skip actually sending equity to the borrower, while
 - still exporting `inputAction.equity` for other validators to use.
- Invariants that should hold:
 - `inputAction.equity` must never be negative.
 - If `partialLiquidationPenaltyPerMille < 0`, equity should not be computed at all; `inputAction.equity` must be `0`, and borrower compensation must be disabled in this branch.
 - If `partialLiquidationPenaltyPerMille >= 0`, then:
 - `inputAction.equity` must be `>= 0` and equal to the corresponding `get_equity*` helper (any negative intermediate result in `get_equity*` must be clamped to `0`), and
 - whenever `inputAction.equity > 0`, the transaction must include a borrower compensation output matching this value.

Because `lm_liquidate_action` consumes `inputAction.equity` without re-checking these invariants, this validator must enforce them. Otherwise, `inputAction.equity` can become a negative or inconsistent value, breaking assumptions in downstream validators and making some liquidations fail.

Relevant snippet (simplified):

```
1 Liquidation {
2   lTV,
3   lTVDivider,
4   equityInPrincipalCurrency,
```

 Aiken

```

5   partialLiquidationPenaltyPerMille,
6 } -> {
7   ...
8   and {
9     // Currently only equality is enforced; sign is not.
10    if equityInPrincipalCurrency {
11      inputAction.equity == get_equity(...)
12    } else {
13      inputAction.equity == get_equity_in_collateral_currency(...)
14    },
15
16    or {
17      isRepaymentLate,
18      can_liquidate(...),
19    },
20
21    or {
22      // If the penalty parameter can ever be negative on-chain, the contract:
23      // - still computes a (boosted) equity amount, but
24      // - does not enforce that this equity is sent to the borrower.
25      //
26      // This value (`inputAction.equity`) is exported for other validators to
27      // use assuming it is correct.
28      partialLiquidationPenaltyPerMille < 0,
29      inputAction.equity <= 0,
30      {
31        let borrowerCompensationOutput = ...
32        equity_sent_to_borrower(...)
33      },
34    },
35  }
36 }

```

Scenario

1. Negative equity with missing non-negativity check

- Borrower takes a loan and deposits collateral token X.
- Due to steep market movements, the borrower's collateral value drops below the loan amount, resulting in negative equity.
- An automated bot triggers liquidation via `lm_liquidate_action`.

- The liquidation validator `loan_claim_action.ak` computes a negative equity value using `get_equity`, but does not enforce non-negativity on `inputAction.equity`.
- The borrower-compensation `or` block is satisfied without requiring any compensation, even though `inputAction.equity` is negative.
- `lm_liquidate_action` uses this negative `loanInputAction.equity`. As a result, it derives `loanCollateralThatLenderShouldReceive > loanCollateralAmount`, which is impossible to fulfill, causing `containsCorrectAmount` to be `false` and the transaction to be rejected.
- The automated liquidation process fails, leaving the lender exposed to further loss in case the collateral price drops further.

2. Case with `partialLiquidationPenaltyPerMille < 0`

- Borrower takes a loan and deposits collateral token Y.
- The borrower's collateral value drops, and the loan enters liquidation.
- The liquidation validator `loan_claim_action.ak` computes a boosted equity value using `get_equity`, but since `partialLiquidationPenaltyPerMille < 0`, it does not enforce any invariants on `inputAction.equity`.
- An inflated `inputAction.equity` is exported, but in reality it should be zero. This results in false or inaccurate equity information that could be misused by other validators or off-chain systems relying on this data.
- Note: currently `lm_liquidate_action` enforces `partialLiquidationPenaltyPerMille > 0` before calculating `loanCollateralThatLenderShouldReceive`, so this scenario does not lead to immediate transaction failure. However, the incorrect equity value can still cause confusion and potential issues in systems that rely on this data.

Recommendation

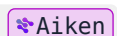
1. Equity invariants around `partialLiquidationPenaltyPerMille`:

- If `partialLiquidationPenaltyPerMille < 0`, skip equity computation entirely and require `inputAction.equity == 0`.
- Otherwise, enforce:
 - `inputAction.equity >= 0`, and
 - `inputAction.equity` equals the result of `get_equity*`, with any negative intermediate result clamped to `0`.

2. Borrower-compensation `or` block:

- Once `partialLiquidationPenaltyPerMille < 0` is handled by forcing `inputAction.equity == 0`, remove the explicit penalty check from the borrower-compensation `or` block.
- Enforce that if equity is positive, it is actually sent to the borrower.

```
1 use aiken/math.{max}
```

 Aiken


```

2  ...
3  // equity non-negativity and correctness condition
4  if partialLiquidationPenaltyPerMille < 0 {
5    // Equity disabled: must be zero, and no borrower compensation.
6    inputAction.equity == 0
7  } else {
8    // Normal case: equity must be >= 0 and correctly paid out if positive.
9    and {
10     inputAction.equity >= 0,
11     if equityInPrincipalCurrency {
12       let computed_equity = get_equity(...)
13       inputAction.equity == max(computed_equity, 0)
14     } else {
15       let computed_equity = get_equity_in_collateral_currency(...)
16       inputAction.equity == max(computed_equity, 0)
17     },
18   }
19 },
20 ...
21 // borrower compensation / equity payout condition
22 or {
23   inputAction.equity == 0,
24   {
25     let borrowerCompensationOutput = ...
26     equity_sent_to_borrower(...)
27   },
28 },

```

By enforcing these invariants, we ensure that `inputAction.equity` is always a valid, non-negative value that accurately reflects the borrower's equity, maintaining consistency across validators that rely on this field.

Resolution

Resolved by applying the recommended fix.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[H 402] Incorrect Usage of maxAllowedSlippage

	Level	Severity	Status
	4	High	Resolved

Category: Parameter Misconfiguration

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

`maxAllowedSlippage` is declared as an `Int`, but the computed `slippage` is a fractional ratio:

```
1  validator actionValidator(
2      ...
3      maxAllowedSlippage: Int,
4      ...
5  ) {...}
6
7  fn check_slippage_after_swap(price_x_final, price_x_init, maxAllowedSlippage) {
8      //slippage = price_x_final / price_x_init - 1.0
9      expect Some(priceRatio) = rational.div(price_x_final, price_x_init)
10     let slippage = rational.sub(priceRatio, rational.from_int(1))
11     rational.compare(
12         rational.abs(slippage),
13         rational.from_int(maxAllowedSlippage),
14     ) == Less
15 }
```

In most cases, the calculated `slippage` will be a fractional number (e.g., `+/-0.05` for 5% slippage). However, `maxAllowedSlippage` is an integer value (`0`, `1`, `2` etc). This means that the check will always pass for any slippage below `100%`, which is likely not the intended behavior. High slippage values can lead to significant losses for users executing swaps.

Recommendation

Setup `maxAllowedSlippage` to represent basis points (bps) where `1 bp = 0.01%` (i.e., `1 = 0.01%`, `100 = 1%`, `10000 = 100%`) and update the comparison to account for this scaling:

```
1  rational.compare(
2      rational.abs(slippage),
3      rational.div(rational.from_int(maxAllowedSlippage), rational.from_int(10000)),
4  ) == Less
```

*Note: If Option 2 of the recommendation from Finding **F403** is implemented, this issue is no longer relevant as `maxAllowedSlippage` would be removed from the validator.*

Resolution

Resolved by implementing Option 2 of the recommendation from Finding **F403**, which removes the `maxAllowedSlippage` parameter from the validator.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[H 403] Inadequate Slippage Protection

	Level	Severity	Status
	4	High	Resolved

Category: Slippage Protection / External Protocol Interaction

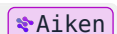
Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

To convert collateral into the debt asset, `lm_liquidate_and_convert_action` creates a Minswap order using `SwapExactIn`. However, the order is constructed with `minimum_receive` hardcoded to `1`:

To convert the collateral to the debt asset, the contract uses minswap's swap functionality. However, it sets the `minimum_receive` parameter to a fixed value of `1`, which can lead to significant slippage and potential loss of funds during the swap process.

```
1 let orderType =
2   minswap_types.SwapExactIn {
3     a_to_b_direction: lpABDirection,
4     swap_amount_option: minswap_types.SAOSpecificAmount {...},
5     minimum_receive: 1,
6     killable: True,
7   }
```

 Aiken

`minimum_receive` is the only on-chain lower bound that Minswap enforces at order execution time. Setting it to `1` effectively disables slippage protection for this conversion.

Although the `lm_liquidate_and_convert_action` validator computes an expected output (`amountOut`) and applies slippage checks on that computed value (and compare it against the remaining debt), these checks do not influence the Minswap order execution due to the hardcoded `minimum_receive`.

At execution time, Minswap calculates the actual output amount based on the current pool reserves and fees using its internal `math.calculate_amount_out` function. Since the `minimum_receive` is set to `1`, Minswap will execute the swap as long as it can provide at least `1` unit of the debt asset, regardless of how much slippage or price movement occurs compared to the expected `amountOut` calculated earlier.

An executor could exploit this by executing the swap when the pool conditions are highly unfavorable (e.g., due to price movement, low liquidity, or manipulation).

As a result, the protocol can end up selling collateral for far less debt asset than intended, potentially leaving the liquidation underfunded and causing significant loss of value during conversion.

Since `amountOut` is not guaranteed to reflect the actual received amount, the check `amountOut >= finance.get_remaining_debt(..)` is insufficient to ensure that the liquidation covers the debt.

Recommendation

OPTION 1: Calculate `minimum_receive` based on acceptable slippage

Note: `maxAllowedSlippage` represents total price variation tolerated during the swap, including price impact, fees, etc., expressed in basis points (1/10,000). For example, a value of `50` corresponds to a maximum slippage of `0.5%`.

1. Calculate the `minimum_receive` parameter dynamically based on acceptable slippage thresholds.

```

1  expect Some(currentPriceCollateral) = rational.new(reservePrincipal,
2  expect Some(slippageFraction) = rational.new(maxAllowedSlippage, 10_000)
3  let worstAcceptablePrice =
4    rational.mul(
5      currentPriceCollateral,
6      rational.sub(rational.from_int(1), slippageFraction),
7    )
8  let minimumReceive =
9    rational.floor(
10     rational.mul(
11       worstAcceptablePrice,
12       rational.from_int(swappableCollateralAmount),
13     ),
14   )
15  let orderType =
16    minswap_types.SwapExactIn {
17      a_to_b_direction: lpABDirection,
18      swap_amount_option: minswap_types.SAOSpecificAmount {...},
19      minimum_receive: minimumReceive,
20      killable: True,
21    }

```

2. Remove the separate slippage check on `amountOut`:

Since the `minimum_receive` now accounts for slippage, the additional check on `amountOut` is redundant and can be removed to streamline the validation process.

Remove `check_slippage_after_swap`:

```
1 check_slippage_after_swap(
2     collateralPriceIn,
3     collateralPriceOut,
4     maxAllowedSlippage,
5 ),
```

 Aiken

Remove the calculation of `collateralPriceIn` and `collateralPriceOut`:

```
1 expect Some(collateralPriceIn) =
2     rational.new(reserveCollateral, reservePrincipal)
3 expect Some(collateralPriceOut) =
4     rational.div(
5         rational.from_int(reservePrincipal - amountOut),
6         rational.from_int(reserveCollateral + swappableCollateralAmount),
7     )
```

 Aiken

Remove the function definition of `check_slippage_after_swap`:

```
1 fn check_slippage_after_swap(price_x_final, price_x_init, maxAllowedSlippage)
2 {..}
```

 Aiken

3. Use `minimumReceive` directly for debt comparison:

Since `minimum_receive` now reflects the worst-case acceptable output after slippage, it guarantees that the received amount will cover the debt if the swap executes.

Replace `amountOut` with `minimumReceive` in debt comparison:

```
1 // Replace this
2 amountOut >= finance.get_remaining_debt(..)
3
4 // With this
5 minimumReceive >= finance.get_remaining_debt(..)
```

 Aiken

4. Remove the calculation of `amountOut` and its dependent code:

```
1 let diff = minswapV2DefaultTradingFeeDenominator - tradingFeeNumerator
2 let inWithFee = diff * swappableCollateralAmount
3 let numerator = inWithFee * reservePrincipal
4 let denominator =
5     minswapV2DefaultTradingFeeDenominator * reserveCollateral + inWithFee
6 expect Some(amountOut) = rational.new(numerator, denominator)
```

 Aiken

Remove the parameter `minswapV2DefaultTradingFeeDenominator` from the validator function signature:

```
1 validator actionValidator(
2     ...
3     minswapV2DefaultTradingFeeDenominator: Int,
4     ...
5 )
```

 Aiken

Remove the calculation of `tradingFeeNumerator`:

```
1 let tradingFeeNumerator =
2     if isCollateralAssetA {
3         base_fee_a_numerator
4     } else {
5         base_fee_b_numerator
6     }
```

 Aiken

Remove the extraction of `base_fee_a_numerator` and `base_fee_b_numerator` from the pool datum:

```
1 expect PoolDatum {
2     // Keep other fields
3     asset_a,
4     asset_b,
5     reserve_a,
6     reserve_b,
7     // Remove these two lines
8     base_fee_a_numerator,
9     base_fee_b_numerator,
10    ..
11 } = minswapPoolRefInputDatum
```

 Aiken

OPTION 2: Set `minimum_receive` to the remaining debt directly (PREFERRED)

1. Set `minimum_receive` directly to the remaining debt amount:

```
1 let remainingDebt = finance.get_remaining_debt(..)
2 let orderType =
3     minswap_types.SwapExactIn {
4         a_to_b_direction: lpABDirection,
5         swap_amount_option: minswap_types.SAOSpecificAmount {...},
6         minimum_receive: remainingDebt,
7         killable: True,
8     }
```

 Aiken

2. Remove the separate slippage check on `amountOut` :

Since `minimum_receive` now directly ensures that the received amount will cover the debt, the additional check on `amountOut` is redundant and can be removed to streamline the validation process.

Remove `check_slippage_after_swap` :

```
1 check_slippage_after_swap(✖ Aiken
2     collateralPriceIn,
3     collateralPriceOut,
4     maxAllowedSlippage,
5 ),
```

Remove the calculation of `collateralPriceIn` and `collateralPriceOut` :

```
1 expect Some(collateralPriceIn) = ✖ Aiken
2     rational.new(reserveCollateral, reservePrincipal)
3 expect Some(collateralPriceOut) =
4     rational.div(
5         rational.from_int(reservePrincipal - amountOut),
6         rational.from_int(reserveCollateral + swappableCollateralAmount),
7     )
```

Remove the function definition of `check_slippage_after_swap` :

```
1 fn check_slippage_after_swap(price_x_final, price_x_init, maxAllowedSlippage) ✖ Aiken
2     {...}
```

3. Remove the debt comparison using `amountOut` and rely on `minimum_receive` :

Since `minimum_receive` is set to the remaining debt, it guarantees that the received amount will cover the debt if the swap executes.

Remove the debt comparison using `amountOut` :

```
1 amountOut >= finance.get_remaining_debt(..) ✖ Aiken
```

4. Remove the calculation of `amountOut` and its dependent code:

```
1 let diff = minswapV2DefaultTradingFeeDenominator - tradingFeeNumerator ✖ Aiken
2 let inWithFee = diff * swappableCollateralAmount
3 let numerator = inWithFee * reservePrincipal
4 let denominator =
5     minswapV2DefaultTradingFeeDenominator * reserveCollateral + inWithFee
6 expect Some(amountOut) = rational.new(numerator, denominator)
```


Remove the parameters `minswapV2DefaultTradingFeeDenominator` and `maxAllowedSlippage` from the validator function signature:

```
1 validator actionValidator(
2     ...
3     minswapV2DefaultTradingFeeDenominator: Int,
4     maxAllowedSlippage: Int,
5     ...
6 )
```

Remove the calculation of `tradingFeeNumerator`:

```
1 let tradingFeeNumerator =
2     if isCollateralAssetA {
3         base_fee_a_numerator
4     } else {
5         base_fee_b_numerator
6     }
```

Remove the extraction of `base_fee_a_numerator`, `base_fee_b_numerator`, `reserve_a`, and `reserve_b` from the pool datum:

```
1 expect PoolDatum {
2     // Keep other fields
3     asset_a,
4     asset_b,
5     // Remove these four lines
6     reserve_a,
7     reserve_b,
8     base_fee_a_numerator,
9     base_fee_b_numerator,
10    ..
11 } = minswapPoolRefInputDatum
```

The recommended changes will ensure that slippage protection is effectively enforced during collateral conversion, mitigating the risk of significant losses due to unfavorable swap conditions.

Resolution

Resolved by implementing Option 2 from the recommendation section

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[H 404] Incorrect Calculation of Collateral Price

	Level	Severity	Status
	4	High	Resolved

Category: Calculation Error

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

In the `lm_liquidate_and_convert_action.ak` file, there is a miscalculation of collateral prices. The code inverts the calculation of `collateralPriceIn`, which could lead to incorrect slippage checks. Specifically, the calculation of `collateralPriceIn` is reversed, which may result in an inaccurate assessment of whether the slippage is within acceptable limits.

```
1 expect Some(collateralPriceIn) =
2   rational.new(reserveCollateral, reservePrincipal) ak
```

Calculating `collateralPriceIn` should use `reservePrincipal` as the numerator and `reserveCollateral` as the denominator to accurately reflect the price of collateral in terms of principal.

Impact

Depending on the actual numbers involved, this issue could lead to situations where the slippage check either passes when it should fail or fails when it should pass. This miscalculation can have significant consequences during the liquidation process. If the validator incorrectly approves a liquidation due to faulty slippage checks, lenders may end up losing funds during the swap, as they might be sending an order that results in high slippage. Conversely, if the validator fails the liquidation when it should have passed, it could lead to under-collateralized loans and financial losses for lenders.

Recommendation

Fix the calculation of `collateralPriceIn` to ensure it accurately reflects the intended price before the swap operation:

```
1 expect Some(collateralPriceIn) =
2   rational.new(reservePrincipal, reserveCollateral) ak
```

*Note: If the recommendation from Finding **F403** is implemented, this issue is no longer relevant.*

Resolution

Resolved by implementing the recommendation from Finding **F403**

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[H 405] Collateral Neglect in Compound Action

	Level	Severity	Status
	4	High	Resolved

Category: Value accounting

Affected Files:

- `validators/lender-manager/lm_compound_action.ak`
- `validators/lender-manager/lm_liquidate_pay_in_advance_and_compound_action.ak`

Description

In the `lm_compound_action.ak` and `lm_liquidate_pay_in_advance_and_compound_action.ak` files, the current implementation calculates the total assets to compound for each pool manager by summing the principal amounts held in the input assets. However, this calculation considers only the principal asset and neglects any additional collateral assets present in the input UTxOs. This oversight can lead to situations in which collateral assets are effectively ignored during compounding, potentially allowing an attacker to exploit them.

Attack Scenario

1. Conditions for Exploitation: For the vulnerability to be exploited, there must exist asset inputs that hold liquidated loans as collateral.
2. Taking Advantage of Neglected Collateral: An attacker could include these asset inputs in a compounding transaction. Since the current implementation only sums the principal asset amounts, any collateral assets present in these inputs can be routed to the attacker without being accounted for in the compounding calculation.

In such a scenario, the calculated asset amount for compounding would be either zero (if the principal asset is not present) or a minimal amount (if the principal asset is ADA, it would only count the min-ADA held by the UTxO). This allows the attacker to siphon off the collateral assets during the compounding process.

Impact

This vulnerability could result in significant financial losses for the lender, as attackers may siphon off collateral assets during the compounding process.

Recommendation

Add validation checks in the asset input processing logic to ensure the following conditions are met:

1. If the principal asset is ADA, ensure that no other tokens are present in the asset input (except for an optional repaymentReceipt NFT).
2. If the principal asset is not ADA, ensure that only the min-ADA, the principal token, and an optional repaymentReceipt NFT are present in the asset input.

(In the code, this should be implemented in the section where `totalAssetsToCompoundForEachPoolManager` is calculated, as shown below:)

```

1 // Get the repayment policy ID from the configuration: to be used for
  validating the presence of the optional repaymentReceipt NFT
2 let repaymentPolicyId = builtin.un_b_data(utils.safe_list_at(config, 7))
3 ...
4 let totalAssetsToCompoundForEachPoolManager =
5   list.indexed_foldr(
6     lmAssetsInputs,
7     ...
8     fn(index, assetInput, result) {
9       ...
10      let assetAmount =
11        quantity_of(
12          assetInput.output.value,
13          principalAsset.policyId,
14          principalAsset.assetName,
15        )
16
17      // New validation checks:
18
19      // 1. If the principal asset is ADA, ensure no other tokens are present (except
20      // for an optional repaymentReceipt NFT)
21      // 2. If the principal asset is not ADA, ensure only min-ADA, the principal
22      // token, and an optional repaymentReceipt NFT are present
23      // Note: Use the repaymentPolicyId to check for the presence of the optional
24      // repaymentReceipt NFT
25    },
26  )

```

Resolution

Resolved by implementing the recommended fix as follows:

```

1 let totalAssetsToCompoundForEachPoolManager =
2   list.indexed_foldr(
3     lmAssetsInputs,
4     ...
5     fn(index, assetInput, result) {
6       ...
7       let receiptNFTCount =
8         utils.quantity_of_policy_id(

```

```
9      assetInput.output.value,  
10      repaymentPolicyId,  
11      expect  
12      if principalAsset.policyId == ada_policy_id {  
13          // 1. If principal asset is ADA, ensure no other tokens are present, except  
          eventual receipt NFT  
14          list.length(flatten(assetInput.output.value)) == 1 + receiptNFTCount  
15      } else {  
16          // 2. If principal asset is not ADA, ensure only min-ADA and principal token  
          are present, except eventual receipt NFT  
17          list.length(flatten(assetInput.output.value)) == 2 + receiptNFTCount  
18      }  
19      ...
```

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[M 301] Currency Unit Mismatch 2

	Level	Severity	Status
	3	Medium	Resolved

Category: Value accounting / Unit mismatch

Affected Files:

- `validators/lender-manager/lm_liquidate_action.ak`
- `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

During automated liquidation, the lender's collateral share is computed by subtracting `loanInputAction.equity` from `loanCollateralAmount` when `partialLiquidationPenaltyPerMille > 0`.

However, this logic implicitly assumes `loanInputAction.equity` is denominated in collateral currency, while `loanInputAction.equity` can be denominated in principal currency. Since `loanCollateralAmount` is always in collateral currency, subtracting equity expressed in principal currency is a unit mismatch and produces an incorrect result.

The relevant code snippet is:

```

1 // at validators/lender-manager/lm_liquidate_action.ak
2 let loanCollateralThatLenderShouldReceive =
3   if partialLiquidationPenaltyPerMille > 0 {
4     loanCollateralAmount - loanInputAction.equity - liquidationFee
5   } else {
6     loanCollateralAmount - liquidationFee
7   }
8
9 // at validators/lender-manager/lm_liquidate_and_convert_action.ak
10 let swappableCollateralAmount =
11   if partialLiquidationPenaltyPerMille > 0 {
12     loanCollateralAmount - loanInputAction.equity - liquidationFee
13   } else {
14     loanCollateralAmount - liquidationFee
15   }

```

While the validator includes these checks:

```

1 // at validators/lender-manager/lm_liquidate_action.ak
2 // Lender bond must NOT allow conversion to principal
3 shouldLiquidationConvertToPrincipal == False
4
5 // at validators/lender-manager/lm_liquidate_and_convert_action.ak

```

```
6 //Lender bond must allow conversion to principal
7 shouldLiquidationConvertToPrincipal,
```

This field does not guarantee that `equityInPrincipalCurrency == False`. There is no on-chain enforced relationship between `shouldLiquidationConvertToPrincipal` and `equityInPrincipalCurrency`. And logically they are independent settings.

As a result, `equityInPrincipalCurrency` can be `True`, regardless of the value of `shouldLiquidationConvertToPrincipal`, leading to the unit mismatch issue.

Impact

The outcome depends on the price ratio between collateral and principal currencies.

Let: `ratio = collateral price / principal price`

When `ratio > 1`,

- `equity_principal > equity_collateral`
- `actual_equity > expected_equity`
- `calculated_lender_collateral < expected_lender_collateral`
- Lender receives less collateral than they are entitled to.

When `ratio < 1`,

- `equity_principal < equity_collateral`
- `actual_equity < expected_equity`
- `calculated_lender_collateral > expected_lender_collateral`
- `validators/loan/loan_claim_action.ak`'s validator enforces borrower to receive the right amount.
- Transaction fails due to unbalanced outputs.

Where:

- `equity_principal` is the equity expressed in principal currency
- `equity_collateral` is the equity expressed in collateral currency
- `actual_equity` is the equity value used in the calculation (i.e., `loanInputAction.equity`)
- `expected_equity` is the correct equity value that should have been used
- `calculated_lender_collateral` is the lender collateral amount computed by the script (i.e., `loanCollateralThatLenderShouldReceive` or `swappableCollateralAmount`)
- `expected_lender_collateral` is the correct lender collateral amount that should have been computed

Attack Scenario:

1. Lender creates a pool with the following Comditions:
 - `equityInPrincipalCurrency = True` at `PoolDatum.commonData.LiquidationMode`
 - `partialLiquidationPenaltyPerMille > 0` at `PoolDatum.commonData.LiquidationMode`

- Collateral currency is more expensive than principal currency (i.e., `ratio > 1`).
2. Borrower takes a loan from that pool.
 3. Loan reaches liquidation conditions.
 4. Borrower executes automated liquidation, using `lm_liquidate_action.ak` or `lm_liquidate_and_convert_action.ak` validator action.
 5. Due to the currency mismatch, the script computes a smaller `loanCollateralThatLenderShouldReceive / swappableCollateralAmount` than intended.
 6. Lender receives less collateral than they are entitled to. And the rest of the collateral are returned to the borrower.

Recommendation

Ensure the equity subtracted from `loanCollateralAmount` is always in collateral currency.

This is achieved by adding the following check:

```
1 expect equityInPrincipalCurrency == false
```

 Aiken

Suggested patch:

```
1 ...
2 validator actionValidator(...) {
3   withdraw(...) {
4     ...
5     let liquidatedCollateralOutputsAreCorrect =
6       list.indexed_foldr(
7         loanInputs,
8         True,
9         fn(index, loanInput, result) {
10           ...
11           expect Liquidation {
12             partialLiquidationPenaltyPerMille,
13 +           equityInPrincipalCurrency,
14             ..
15           } = loanInputAction.liquidationMode
16 +           expect equityInPrincipalCurrency == False
17           ...
18         }
19       )
20     ...
21   }
22 }
```

 diff

Resolution

Resolved by adding validation to ensure `equityInPrincipalCurrency` is `False`

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[M 302] Inversion of Arguments in Slippage Check

	Level	Severity	Status
	3	Medium	Resolved

Category: Function Argument Error

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

In the `lm_liquidate_and_convert_action.ak` file, there is an inversion of parameters when calling the slippage check function `check_slippage_after_swap`. The current implementation incorrectly uses `collateralPriceIn` as the final price and `collateralPriceOut` as the initial price, which leads to an incorrect calculation of slippage. This inversion can result in a slightly higher calculated slippage than intended, potentially leading to failed transactions during automated liquidations.

```
1 check_slippage_after_swap(
2     collateralPriceIn,
3     collateralPriceOut,
4     maxAllowedSlippage,
5 ),
```

Aiken

Recommendation

To resolve this issue, the parameters passed to the `check_slippage_after_swap` function should be inverted. The correct implementation should use `collateralPriceOut` as the final price and `collateralPriceIn` as the initial price. The corrected code should look like this:

```
1 check_slippage_after_swap(
2     collateralPriceOut,
3     collateralPriceIn,
4     maxAllowedSlippage,
5 ),
```

Aiken

This change will ensure that the slippage is calculated correctly, aligning with the intended logic of the function and preventing unnecessary transaction failures during liquidations.

*Note: If the recommendation from Finding **F403** is implemented, this issue is no longer relevant.*

Resolution

Resolved by implementing the recommendation from Finding **F403**

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[M 303] Unhandled Asset Conversion Failures

	Level	Severity	Status
	3	Medium	Resolved

Category: Smart Contract Logic / Fund Recovery

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

In the `lm_liquidate_and_convert_action.ak` validator, if the asset conversion process fails, there are no fallback mechanisms in place to handle this failure. The current implementation does not provide options to retrieve failed conversions or to retry the conversion process.

This oversight can lead to situations where the UTxO becomes locked within the lender manager smart contract, preventing lenders from recovering their funds. Specifically, the `lm_withdraw_assets_action.ak` validator expects certain datum structures that will not be met if the conversion fails, resulting in an inability to proceed with fund recovery.

```

1  let inputDatumHash =
2    crypto.blake2b_256(builtin.serialise_data(loanInput))
3
4  ...
5
6  let expectedOrderDatum =
7    InlineDatum(
8      minswap_types.OrderDatum {
9        ...
10       refund_receiver: lenderBondInput.output.address,
11       refund_receiver_datum: minswap_types.EODInlineDatum {
12         hash: inputDatumHash,
13       },
14       ...
15     },
16   )

```



Recommendation

To resolve this issue, it is recommended to implement one or both of the following strategies:

1. **Default to Liquidation-Only Action:** In the event of a conversion failure, the returned UTxO should allow lenders to recover their funds without relying on the conversion process.

- To achieve this, we need to modify the datum hash calculation to reflect a liquidation-only action when conversion fails. This involves creating a datum that matches the expected structure for liquidation without conversion.

```

1  // rename `inputDatumHash` to `refundDatumHash` for clarity
2  let refundDatumHash =
3    RepaymentDatumWithToken {
4      inputOutputReference: loanInput.output_reference,
5      action: constants.claimed_collateral_action,
6      data: None,
7      ownerAsset: ownerAsset,
8    }
9    |> builtin.serialise_data
10   |> crypto.blake2b_256
11   ...
12  let expectedOrderDatum =
13    InlineDatum(
14      minswap_types.OrderDatum {
15        ...
16        refund_receiver: lenderBondInput.output.address,
17        refund_receiver_datum: minswap_types.EODInlineDatum {
18          hash: refundDatumHash,
19        },
20        ...
21      },
22    )

```

2. **Retry Conversion:** Implement a retry mechanism that attempts the conversion process multiple times before ultimately defaulting to a liquidation-only route. This could involve setting a maximum number of retries and incorporating delays between attempts.
 1. Implement a convert action that can handle returned UTxOs from failed conversions.
 2. Set the datum hash accordingly to reflect the retry logic.

(Conceptual implementation)

```

1  // rename `inputDatumHash` to `refundDatumHash` for clarity
2  let refundDatumHash =
3    RepaymentDatumWithToken {
4      inputOutputReference: loanInput.output_reference,
5      action: constants.unconverted_collateral_action,
6      data: ConversionRetryData {
7        remainingRetries: MAX_RETRY_LIMIT,

```

```
8      timestamp: validFrom,  
9      delayBetweenRetries: RETRY_DELAY,  
10     orderDatum: expectedOrderDatum,  
11   },  
12   ownerAsset: ownerAsset,  
13 }  
14 |> builtin.serialise_data  
15 |> crypto.blake2b_256  
16 // MAX_RETRY_LIMIT and RETRY_DELAY can be protocol defined constants or user defined  
   in the ClaimData (loanInputAction)  
17 // validFrom is derived from self.validity_range.lower_bound.bound_type
```

Note: This implementation would require defining new datum structure `ConversionRetryData` and constant `constants.unconverted_collateral_action`.

By incorporating these strategies, the system can ensure that lenders have a reliable means of recovering their funds even in the event of conversion failures.

Resolution

- Resolved by implementing strategy 1 from the recommendation section, which defaults to a liquidation-only action in the event of conversion failure.
- Strategy 2 (Retry Conversion) was not implemented.

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[M 304] Negative liquidationFeePerMille Exploit

	Level	Severity	Status
	3	Medium	Resolved

Category: Negative Parameter Exploit / Fund Drain

Affected Files:

- `validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak`
- `validators/lender-manager/lm_liquidate_pay_in_advance_and_compound_action.ak`

Description

In the `lm_liquidate_and_pay_in_advance_action.ak` and `lm_liquidate_pay_in_advance_and_compound_action.ak` validators, there is no validation to ensure that the `liquidationFeePerMille` parameter is non-negative. This oversight allows an attacker to set a negative value for `liquidationFeePerMille`, which can lead to an inflated amount being paid to lenders during the liquidation process.

Since these contracts operate on a pay-in-advance basis, and since these actions are intended to be executed by automated bots, there are multiple scenarios that could happen depending on the pay-in-advance design:

- If the pay-in-advance system is designed such that bots draw funds from their own private pools, and these bots do not implement checks to ensure that the amount paid out is at least equal to the value of collateral they are receiving, then automated bot operations without proper checks could lead to a drain of the bot's private funds.
- Similarly, if these bots are owned by the protocol, and if no checks are implemented, this could lead to a drain of the protocol funds that run these bots.
- If designing a decentralized pay-in-advance system where the bot draws funds from pay-in-advance-specific pools, then if the pay-in-advance scripts don't make sure that what goes out is at least equal to what comes in, then this could also lead to a drain of these pay-in-advance-specific pools.

Here is the relevant code snippet where the issue arises:

(The code snippet is illustrative; actual code may vary)

```

1  expect LenderManagerDatum {
2    shouldLiquidationConvertToPrincipal,
3    liquidationFeePerMille,
4    ..
5  } = inputLenderBondDatum
6  let liquidationFee = loanCollateralAmount * liquidationFeePerMille / 1000
7  let loanCollateralThatLenderShouldReceive = loanCollateralAmount -
    loanInputAction.equity - liquidationFee
8  // let loanCollateralToSend = loanCollateralAmount - liquidationFee
9  let convertedLoanCollateralToPrincipalAmount =

```

 Aiken

```

10  finance.convertFromAToBWithOracles(
11    collateralOraclePriceFeed,
12    principalOraclePriceFeed,
13    rational.from_int(loanCollateralThatLenderShouldReceive),
14    // rational.from_int(loanCollateralToSend),
15  )
16  ...
17  //Ensures lender receives the correct assets, the eventual output to borrower is
    already checked by loan.ak
18  validate_repayment_output(
19    loanInputOutputReference: loanInput.output_reference,
20    repaymentOutput: lmAssetsOutput,
21    lenderBondAsset: Asset {
22      policyId: lenderBondPolicyId,
23      assetName: lenderBondAssetName,
24    },
25    repaymentAsset: Asset(collateralPolicyId, collateralAssetName),
26    repaymentAmount: convertedLoanCollateralToPrincipalAmount,
27    action: constants.converted_to_liquidity_action,
28  ),

```

When `liquidationFeePerMille` is negative, the calculated `liquidationFee` becomes negative, resulting in the `loanCollateralThatLenderShouldReceive/loanCollateralToSend` being larger than intended. Consequently, the `convertedLoanCollateralToPrincipalAmount` withdrawn from the bot's fund exceeds the actual collateral value, leading to a drain of funds from the bot's private pool or the protocol pool, depending on the design of the pay-in-advance system.

Why this is possible

- There are no checks in place to validate that `liquidationFeePerMille` is non-negative before it is used in calculations.
- The `LenderManagerDatum` is user-controlled, allowing an attacker to manipulate the `liquidationFeePerMille` value.
- The automated nature of bot operations means that if an attacker sets a negative `liquidationFeePerMille`, the bot could automatically execute the liquidation process and drain funds without any manual intervention.

Important Note

The protocol need to rely on external checks implemented in the pay-in-advance scripts or bots to ensure that the amount paid out is at least equal to the value of collateral received. However, this is not a robust solution as it relies on external systems and does not enforce

safety at the contract level. Therefore, it is crucial to implement validation checks within the smart contracts themselves to prevent such exploits.

Recommendation

Add validation check `expect liquidationFeePerMille >= 0` in both validators (`lm_liquidate_and_pay_in_advance_action.ak` and `lm_liquidate_pay_in_advance_and_compound_action.ak`) to ensure that the `liquidationFeePerMille` parameter is non-negative before it is used in any calculations. This can prevent attackers from setting a negative liquidation fee and exploiting the protocol pool.

```
1 expect LenderManagerDatum {
2     shouldLiquidationConvertToPrincipal,
3     liquidationFeePerMille,
4     ..
5 } = inputLenderBondDatum
6
7 // Ensure liquidationFeePerMille is non-negative to prevent protocol pool drain
8 expect liquidationFeePerMille >= 0
9
10 // Proceed with the rest of the calculations as previously defined
11 ...
```

 Aiken

Resolution

Resolved by adding validation to ensure `liquidationFeePerMille` is non-negative

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 101] Non-idiomatic list traversal 1

	Level	Severity	Status
	1	Informational	Resolved

Category: Code Quality / Performance Improvement

Affected File: `validators/loan/loan_claim_action.ak`

Description

The validator currently traverses `loanInputs` using `list.indexed_foldr` with a Boolean as an accumulator, and uses `expect` inside the folding function to enforce that previous inputs have passed certain checks:

```
1 list.indexed_foldr(
2   loanInputs,
3   True,
4   fn(index, input, result) {
5     expect result
6     // per-input checks returning Bool
7   },
8 )
```

Aiken

In this use case, there is no meaningful accumulation besides carrying the previous Boolean so that `expect result` can assert that earlier checks succeeded. This effectively reimplements an “indexed all” helper inline. Expressing the logic via a general fold makes the intent less obvious to future readers and encourages duplicating this pattern in other places.

Impact

- The current pattern mixes two enforcement mechanisms: an internal `expect result` (which never checks the last element, since it sees the initial accumulator `True`) and a separate use of the final Boolean result. This asymmetry makes the control flow fragile: if the pattern is copied or refactored and the final result is not properly enforced, the last element’s checks may no longer be guaranteed, despite the presence of `expect result` inside the fold.
- Slightly decreases readability: the logical intent is expressed via a general fold instead of a domain-specific helper.

Recommendation

Introduce a small utility function in `lib/fluidtokens/utils.ak` that captures the pattern explicitly:

```
1 /// Returns `True` if `predicate(index, element)` holds for every element of
   the list.
```

Aiken

```
2  /// Returns `False` as soon as any predicate fails.
3  pub fn indexed_all(self: List<a>, predicate: fn(Int, a) -> Bool) -> Bool {
4      list.indexed_foldr(
5          self,
6          True,
7          fn(index, element, acc) {
8              predicate(index, element) && acc
9          },
10     )
11 }
```

Then, in `validators/loan/loan_claim_action.ak`, replace the manual fold with the new helper:

```
1  utils.indexed_all(  
2    loanInputs,  
3    fn(index, input) {  
4      // existing per-input checks (must return Bool)  
5    },  
6  )
```

Performance Benchmark

```

1 └─ fluidtokens/benchmark.ak ───────────────────────────────────
2 │ PASS [mem: 56.15 K, cpu: 18.03 M] test_current_implementation
3 │ PASS [mem: 55.85 K, cpu: 17.98 M] test_indexed_all
4 └────────────────────────────── 2 tests | 2 passed | 0 failed
```

In this test setup with simplified per-input logic, the refactored implementation (`test_indexed_all`) reduces CPU by 0.05 M and memory by 0.30 K compared to the current implementation.

Additional Occurrences

This pattern also appears in the following validators, which should likewise be updated to use `indexed all`:

- `validators/bond.ak`
- `validators/dutch_auction.ak`
- `validators/pool_manager.ak`
- `validators/pool.ak`
- `validators/repayment.ak`
- `validators/request.ak`
- `validators/conditions/pool_kyc_token.ak`
- `validators/conditions/request specific lenders.ak`

- `validators/lender-manager/lm_compound_action.ak`
- `validators/lender-manager/lm_liquidate_action.ak`
- `validators/lender-manager/lm_liquidate_and_convert_action.ak`
- `validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak`
- `validators/lender-manager/lm_liquidate_pay_in_advance_and_compound_action.ak`
- `validators/lender-manager/lm_withdraw_assets_action.ak`
- `validators/loan/loan_change_collateral_action.ak`
- `validators/loan/loan_recast_action.ak`
- `validators/loan/loan_repay_action.ak`

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 102] Non-idiomatic list traversal 2

Level	Severity	Status
1	Informational	Resolved

Category: Code Quality / Performance Improvement

Affected Files:

- `validators/lender-manager/lm_withdraw_bonds_action.ak`
- `validators/lender-manager/lm_withdraw_assets_action.ak`

Description

The validators currently traverses `lmInputs` and `lenderBondInputs` using `list.foldr` with a Boolean as an accumulator, and uses `expect` inside the folding function to enforce that previous inputs have passed certain checks:

```
1 list.foldr(
2   listToCheck, // the list being traversed (lmInputs or lenderBondInputs)
3   True,
4   fn(input, result) {
5     expect result
6     // per-input checks returning Bool
7   },
8 )
```

Aiken

In this use case, there is no meaningful accumulation besides carrying the previous Boolean so that `expect result` can assert that earlier checks succeeded. Expressing the logic via a general fold makes the intent less obvious to future readers and encourages duplicating this pattern in other places.

Impact

- The current pattern mixes two enforcement mechanisms: an internal `expect result` (which never checks the last element, since it sees the initial accumulator `True`) and a separate use of the final Boolean result. This asymmetry makes the control flow fragile: if the pattern is copied or refactored and the final result is not properly enforced, the last element's checks may no longer be guaranteed, despite the presence of `expect result` inside the fold.
- Slightly decreases readability: the logical intent is expressed via a general fold instead of a domain-specific function.

Recommendation

Replace the manual fold with `list.all`, which is specifically designed to check that all elements in a list satisfy a given predicate. This makes the intent clearer and avoids the need for an accumulator and `expect` calls.

```

1 list.all(
2   listToCheck, // the list being traversed (lmInputs or lenderBondInputs)
3   fn(input: Input) -> Bool {
4     // existing per-input checks (must return Bool)
5   },
6 )

```

 Aiken

Benchmark Results

```

1 ┌ fluidtokens/benchmark ────────────────────────────────────────────────────────────
2 | PASS [mem: 57.57 K, cpu: 18.19 M] test_current_using_foldr
3 | PASS [mem: 57.27 K, cpu: 18.14 M] test_using_list_all
4 └────────────────────────────────────────────────────────────────────────────────── 2 tests | 2 passed | 0 failed

```

In this test setup with simplified per-input logic, using `list.all` reduces CPU by 0.05 M and memory by 0.30 K compared to the current implementation.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 103] ADA policy literal

	Level	Severity	Status
	1	Informational	Resolved

Category: Code quality

Description

ADA is currently detected by comparing the `policyId` directly to a hardcoded empty `ByteArray` literal:

```
1 policyId == ""
```

 Aiken

Aiken's standard library already exposes the ADA policy id as a named constant:

```
1 ada_policy_id: ByteArray = ""
```

 Aiken

Using the empty literal directly is equivalent *performance-wise*, but it hides intent, duplicates knowledge, and diverges from the standard convention used across other Aiken code.

Recommendation

Use the ADA policy ID constant provided by the Aiken library instead of the raw empty literal:

```
1 policyId == assets.ada_policy_id
```

 Aiken

This keeps the semantics identical while improving readability, consistency with the Aiken standard library, and avoiding a magic value in the validator.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 104] Equity Currency Choice

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization / Code Quality

Affected File: `validators/loan/loan_claim_action.ak`

Description

1. Redundant collateral-type check

The following condition is used multiple times to determine whether the collateral is an NFT (1 unit) or Lovelace vs. a non-lovelace fungible token:

```
1 if or {
2     collateralAmount == rational.from_int(1),
3     datum.collateral.maybeAssetName == None,
4 } {...} else {...}
```

 Aiken

This pattern is repeated at different call sites instead of being expressed once via a dedicated flag or configuration field.

2. Equity currency choice not explicit

Since fungible tokens can have low decimal precision, computing equity in collateral currency can be suboptimal, due to rounding loss. In many cases, computing equity in principal currency is preferable, but this choice is not explicitly encoded in the liquidation configuration and must be inferred from local conditions.

Recommendation

1. Introduce an explicit configuration flag to indicate the currency in which equity should be computed.

```
1 // If `true`, equity is computed in principal currency.
2 // If `false`, equity is computed in collateral currency.
3 equityInPrincipalCurrency: Bool,
```

 Aiken

2. Add this field to the LiquidationMode type:

```
1 pub type LiquidationMode {
2     NoLiquidationFullCollateralClaim
3     NoLiquidationDutchAuctionClaim
4     Liquidation {
5         ...
6         equityInPrincipalCurrency: Bool,
7         ...
8     }
```

 Aiken

```
8   }  
9 }
```

3. Replace all the repeated collateral-type checks with this flag.

```
1  if equityInPrincipalCurrency {  
2    ...  
3  } else {  
4    ...  
5  }
```

Aiken

This removes redundant logic, makes the equity currency choice explicit, and allows safer handling of low-precision fungible collateral tokens.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 105] Double traversal during NFT mint validation

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization

Affected Files:

- `validators/lender-manager/lm_config.ak`
- `validators/config.ak`

Description

In the current implementation, the validators check properties of `self.mint` multiple times using helper functions that each traverse the minted assets.

Current pattern:

```
1 // Only 1 token of same policy
2 quantity_of_policy_id(self.mint, policy_id) == 1,
3 // I am minting only 1 token (NFT)
4 quantity_of(self.mint, policy_id, constants.config_asset_name) == 1,
```

 Aiken

Recommendation

Avoid traversing `self.mint` multiple times by restructuring the logic to perform a single traversal of `self.mint`:

1. Extract the token map once: `assets.tokens(self.mint, policy_id)`.
2. Compare the extracted map to the expected dictionary of minted assets (exact asset name and quantity).

```
1 assets.tokens(self.mint, policy_id) == dict.from_pairs(
2     [Pair(constants.config_asset_name, 1)],
3     ),
```

 Aiken

Code Diff (only the relevant snippet is shown):

```
1 + use aiken/collection/dict
2 ...
3 validator config(...) {
4     mint(...) {
5         ...
6         and {
7             ...
8 -         // Only 1 token of same policy
9 -         quantity_of_policy_id(self.mint, policy_id) == 1,
10 -         // I am minting only 1 token (NFT)
```

 diff

```

11 -     quantity_of(self.mint, policy_id, constants.config_asset_name) == 1,
12 +     assets.tokens(self.mint, policy_id) == dict.from_pairs(
13 +         [Pair(constants.config_asset_name, 1)],
14 +     ),
15     ...
16 }
17 }
18 }
```

Benchmark Results

```

1  └─ fluidtokens/benchmark ───────────────────────────────────────────────────
2  | PASS [mem: 156.88 K, cpu: 46.23 M] test_current_double_traversal
3  | PASS [mem: 80.33 K, cpu: 24.61 M] test_single_traversal_optimization
4  └────────────────────────────────────────────────── 2 tests | 2 passed | 0 failed
```

In this test setup, optimizing to a single traversal reduces CPU by 21.62 M and memory by 76.55 K compared to the current implementation (47 % cost reduction).

Resolution

Implemented the recommended optimization in `lm_config.ak` and `config.ak`
 Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 106] Redundant equity branch in calculation

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization / Code quality

Affected Files:

- `validators/lender-manager/lm_liquidate_action.ak`
- `validators/lender-manager/lm_liquidate_and_convert_action.ak`
- `validators/lender-manager/lm_liquidate_and_pay_in_advance_action.ak`

Description

After applying the recommended fix from finding **F401** (which ensures the exported `loanInputAction.equity` is always ≥ 0 and correctly reflects the liquidation configuration), the following pattern becomes redundant:

```
1 let loanCollateralThatLenderShouldReceive = 
2   if partialLiquidationPenaltyPerMille > 0 {
3     loanCollateralAmount - loanInputAction.equity - liquidationFee
4   } else {
5     loanCollateralAmount - liquidationFee
6   }
```

```
1 let swappableCollateralAmount = 
2   if partialLiquidationPenaltyPerMille > 0 {
3     loanCollateralAmount - loanInputAction.equity - liquidationFee
4   } else {
5     loanCollateralAmount - liquidationFee
6   }
```

The branch on `partialLiquidationPenaltyPerMille > 0` is no longer needed: when equity should not be applied, `loanInputAction.equity` is already guaranteed to be `0`. The two branches therefore collapse to the same effective behavior.

Recommendation

Simplify the calculations by always subtracting `loanInputAction.equity` and remove the redundant condition:

```
1 let loanCollateralThatLenderShouldReceive = 
2   loanCollateralAmount - loanInputAction.equity - liquidationFee

1 let swappableCollateralAmount = 
2   loanCollateralAmount - loanInputAction.equity - liquidationFee
```

This keeps the semantics unchanged (under the invariants from Finding **F401**) while reducing branching and improving code clarity.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 107] Code Duplication of Minswap Types and Functions

Level	Severity	Status
1	Informational	Resolved

Category: Code Quality / Maintainability

Affected Files:

- `validators/lender-manager/lm_liquidate_and_convert_action.ak`
- `lib/fluidtokens/types/minswap_v2_types.ak`

Description

The current codebase defines Minswap-related types and functions directly within its own files. In particular, the file `lib/fluidtokens/types/minswap_v2_types.ak` contains type definitions that mirror those found in the official Minswap codebase. The file `validators/lender-manager/lm_liquidate_and_convert_action.ak` also defines Minswap's functionality instead of importing them.

This approach leads to code duplication, which can introduce maintenance challenges and potential inconsistencies if Minswap updates their types or logic.

Moreover, some implementations deviate from the official Minswap definitions:

For instance, the `amountOut` at `validators/lender-manager/lm_liquidate_and_convert_action.ak` is defined as a `rational` type, preserving precision during calculations, whereas Minswap defines `amountOut` as an `int`, and uses integer division, rounding down to the nearest integer.

Recommendation

To enhance code quality and maintainability, it is recommended to import Minswap's type definitions and functions directly from the official Minswap codebase rather than duplicating them. This ensures consistency with Minswap's implementations, simplifies the codebase, and reduces the risk of discrepancies which could lead to unexpected behavior.

To ensure compatibility between the imported Minswap types and the existing codebase, it may be necessary to implement type conversion helper functions.

The helpers would facilitate the usage of Minswap's imported types and functions within the current codebase.

The following steps outline the recommended changes:

1. Add Minswap as a dependency in the project's `aiken.toml` file:

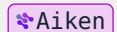
```
1 [[dependencies]]
2 name = "minswap/minswap-dex-v2"
3 version = "v2.1"
4 source = "github"
toml
```

2. Remove the file `lib/fluidtokens/types/minswap_v2_types.ak` entirely.
3. Replace any manually defined Minswap function:
 - At `validators/lender-manager/lm_liquidate_and_convert_action.ak`:
 - import Minswap's module:
 - ▶ `use amm_dex_v2/math as minswap_math`
 - ▶ `use amm_dex_v2/types as minswap_types`
 - ▶ `use amm_dex_v2/utls as minswap_utls`
 - calculate the `amountOut` using Minswap's `calculate_amount_out`.
 - calculate the `lpAssetName` using Minswap's `compute_lp_asset_name`. (Check Finding **F108** for implementation details.)
4. Import Minswap's type definitions and functions directly from the official Minswap codebase.
 - Import Minswap's types wherever they are used: `use amm_dex_v2/types as minswap_types`
5. Add the following helper functions to convert between Minswap types and the existing codebase types:

```

1  // Create a new file `lib/fluidtokens/types/minswap_conversions.ak` and add
  the following code:
2
3  // Conversion functions between our AuthorizationMethod type and minswap's
  OrderAuthorizationMethod type:
4
5  // Convert from AuthorizationMethod to OrderAuthorizationMethod (Minswap)
6  pub fn to_order_auth_method(
7    auth_method: AuthorizationMethod,
8  ) -> minswap_types.OrderAuthorizationMethod {
9    when auth_method is {
10      CardanoSignature { hash } ->
11        minswap_types.OAMSignature { pub_key_hash: hash }
12      CardanoSpendScript { hash } ->
13        minswap_types.OAMSpendScript { script_hash: hash }
14      CardanoWithdrawScript { hash } ->
15        minswap_types.OAMWithdrawScript { script_hash: hash }
16      CardanoMintScript { hash } ->
17        minswap_types.OAMMintScript { script_hash: hash }
18    }
19  }
20
21 // Convert from OrderAuthorizationMethod (Minswap) to AuthorizationMethod
22 pub fn from_order_auth_method(

```



```

23   order_auth_method: minswap_types.OrderAuthorizationMethod,
24 ) -> AuthorizationMethod {
25   when order_auth_method is {
26     minswap_types.OAMSignature { pub_key_hash } ->
27       CardanoSignature { hash: pub_key_hash }
28     minswap_types.OAMSpendScript { script_hash } ->
29       CardanoSpendScript { hash: script_hash }
30     minswap_types.OAMWithdrawScript { script_hash } ->
31       CardanoWithdrawScript { hash: script_hash }
32     minswap_types.OAMMintScript { script_hash } ->
33       CardanoMintScript { hash: script_hash }
34   }
35 }
36
37 // Conversion functions between our Asset type and minswap's Asset type:
38
39 // Convert from minswap_types.Asset to our Asset type
40 pub fn from_minswap_asset(minswapAsset: minswap_types.Asset) -> Asset {
41   Asset { policyId: minswapAsset.policy_id, assetName: minswapAsset.asset_name }
42 }
43
44 // Convert from our Asset type to minswap_types.Asset
45 pub fn to_minswap_asset(asset: Asset) -> minswap_types.Asset {
46   minswap_types.Asset { policy_id: asset.policyId, asset_name: asset.assetName }
47 }

```

More conversion helpers may be needed depending on the specific types and functions used from Minswap.

6. Use the conversion helpers wherever necessary to ensure compatibility between the imported Minswap types and the existing codebase.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 108] Duplicate Implementation of Minswap Functionality

Level	Severity	Status
1	Informational	Resolved

Category: Code Quality / Maintainability / Optimization

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

The current implementation of the `lm_liquidate_and_convert_action.ak` validator manually defines the logic for calculating the Minswap LP asset name.

```

1  let orderedAssetUnits =
2    list.sort(
3      [collateralAssetUnit, principalAssetUnit],
4      bytearray.compare,
5    )
6  let lpAssetName =
7    get_minswap_lp_asset_name(
8      safe_list_at(orderedAssetUnits, 0),
9      safe_list_at(orderedAssetUnits, 1),
10   )

```

```

1  pub fn get_minswap_lp_asset_name(
2    asset_a_unit: ByteArray,
3    asset_b_unit: ByteArray,
4  ) -> ByteArray {
5    let asset_a_ident = crypto.sha3_256(asset_a_unit)
6    let asset_b_ident = crypto.sha3_256(asset_b_unit)
7    let pair_ident = bytearray.concat(asset_a_ident, asset_b_ident)
8    crypto.sha3_256(pair_ident)
9  }

```

This logic duplicates the functionality provided by Minswap's official utility function `compute_lp_asset_name`. This duplication can lead to maintenance challenges, as any updates or changes in Minswap's logic would need to be manually reflected in fluid-tokens' codebase. Additionally, duplicating code increases the risk of inconsistencies and potential bugs.

Moreover, the manual implementation includes a sorting step for asset units, which introduces additional computational overhead. This sorting is unnecessary since the Minswap pool reference input datum already contains the asset units in the correct order. And since the validator enforces that the pool's assets match the protocol's collateral and principal assets, we can use pool assets instead.

Recommendation

To improve code maintainability and reduce redundancy, it is recommended to replace the manual implementation of the LP asset name calculation with Minswap's official `compute_lp_asset_name` utility function.

The following steps outline the recommended changes:

1. Import Minswap's utility module in `lm_liquidate_and_convert_action.ak`:

```
1 use amm_dex_v2/utils as minswap_utils
```

 Aiken

2. Replace the existing LP asset name calculation with a call to Minswap's `compute_lp_asset_name`, using the asset units from the Minswap pool reference input datum:

```
1 let lpAssetName =
2   minswap_utils.compute_lp_asset_name(
3     asset_a.policy_id,
4     asset_a.asset_name,
5     asset_b.policy_id,
6     asset_b.asset_name,
7   )
```

 Aiken

Here, `asset_a` and `asset_b` are obtained from the Minswap pool reference input datum, ensuring they are already in the correct order.

3. Remove the following code from `lm_liquidate_and_convert_action.ak`:
 1. Remove the `get_minswap_lp_asset_name` function entirely from the codebase.
 2. Remove any variables related to the manual calculation, such as `orderedAssetUnits`, `collateralAssetUnit`, and `principalAssetUnit`.
 3. Remove any imports that are no longer needed, such as `use aiken/primitive/bytearray`.

By implementing these changes, the codebase will benefit from improved maintainability, reduced redundancy, and alignment with Minswap's official implementations.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 109] Code Optimization in `lm_liquidate_and_convert_action.ak`

Level	Severity	Status
1	Informational	Resolved

Category: Code Quality / Optimization

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

In the `lm_liquidate_and_convert_action.ak` file, there are opportunities to optimize the code by reducing redundancy and improving clarity.

```
1 let isCollateralAssetA = asset_a == to_minswap_asset(collateralAsset)
2 let lpABDirection =
3   if isCollateralAssetA {
4     True
5   } else {
6     False
7   }
```

Here, the variable `lpABDirection` can be directly assigned the result of the comparison, eliminating the need for the conditional statement.

```
1 //Ensure the minswap pool is for the correct assets
2 if isCollateralAssetA {
3   asset_a == to_minswap_asset(collateralAsset) && asset_b == to_minswap_asset(
4     principalAsset,
5   )
6 } else {
7   asset_a == to_minswap_asset(principalAsset) && asset_b == to_minswap_asset(
8     collateralAsset,
9   )
10 },
```

Here, the first condition in the `if` statement is redundant since it has already been checked above. Merging these two conditions into a single check using `lpABDirection` would enhance code efficiency.

Recommendation

1. Refactor the assignment of `lpABDirection` to directly use the comparison result.

```
1 let lpABDirection = asset_a == to_minswap_asset(collateralAsset)
```

2. Merge the conditions in the asset verification logic to eliminate redundancy and improve clarity.

```
1 if lpABDirection {  
2     asset_b == to_minswap_asset(principalAsset)  
3 } else {  
4     asset_b == to_minswap_asset(collateralAsset) && asset_a ==  
       to_minswap_asset(principalAsset)  
5 },
```

 Aiken

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 110] Redundant Calculation of Remaining Debt

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization

Affected File: `validators/lender-manager/lm_liquidate_and_convert_action.ak`

Description

In the `lm_liquidate_and_convert_action.ak` validator, the remaining debt is recalculated using various parameters from the loan. However, this value is already computed in the `loan_claim_action.ak` validator. By modifying the data structure to include the remaining debt in the `loanInputAction`, we can avoid redundant calculations and improve efficiency.

Recommendation

1. Modify the `ClaimData` structure to include a new field for `remainingDebt` of type `Int`.

```
1 pub type ClaimData {
2   ...
3   // New field to store remaining debt
4   remainingDebt: Int,
5   ...
6 }
```

 Aiken

2. Update the `loan_claim_action.ak` validator to validate the `remainingDebt` field.

```
1 let calculatedRemainingDebt = finance.get_remaining_debt(..)
2 expect calculatedRemainingDebt == inputAction.remainingDebt
3 let remainingDebtAsRational = rational.from_int(calculatedRemainingDebt)
```

 Aiken

3. Refactor the `lm_liquidate_and_convert_action.ak` validator to retrieve the remaining debt directly from the `loanInputAction` instead of recalculating it.

```
1 let minimumReceive = loanInputAction.remainingDebt
```

 Aiken

or (directly in the order creation):

```
1 let orderType =
2   minswap_types.SwapExactIn {
3     ...
4     minimum_receive: loanInputAction.remainingDebt,
5     ...
6   }
```

 Aiken

4. Remove any unused code (variables and imports) that are no longer necessary at `lm_liquidate_and_convert_action.ak` due to this change.

By implementing these changes, we can optimize the liquidation process by eliminating redundant calculations, thereby improving the overall efficiency of the system.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 111] Lack of Unit Tests and Property-Based Tests

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Testing and Quality Assurance

Affected Files:

Affected Components: Fluid Tokens Codebase

Description

The Fluid Tokens codebase currently lacks unit tests and property-based tests. Given the complexity of calculations involved, the absence of these tests may lead to undetected edge cases and potential bugs.

Recommendation

It is recommended to implement comprehensive unit tests to cover individual functions and components within the Fluid Tokens codebase. Additionally, property-based tests should be introduced to validate the behavior of functions across a wide range of inputs, ensuring robustness and reliability in calculations. This will help identify edge cases and improve overall code quality.

[I 112] Documentation Typographical Errors and Inaccuracies

Level	Severity	Status
1	Informational	Resolved

Category: Documentation

Affected Files:

- lib/fluidtokens/types/lender_manager.ak
- lib/fluidtokens/types/loan.ak
- validators/loan/loan_change_collateral_action.ak
- lib/fluidtokens/constants.ak
- lib/fluidtokens/types/minswap_v2_types.ak

Description

In Fluid Tokens, comments are used to provide context and explanations for various tokens and their intended usage. However, some comments contain typographical errors or inaccuracies that could lead to misunderstandings or misinterpretations of the token's purpose.

> lib/fluidtokens/types/lender_manager.ak (line 25)

```
1 //If "", no compounding is allowed
```

ak

Corrected to:

```
1 //If "", no compounding is allowed
```

ak

> lib/fluidtokens/types/loan.ak (line 82)

```
1 //Only for partialLiquidation, it's collateralValue - remainingDebt expressed in
principal currency
```

ak

Corrected to:

```
1 //In case of partialLiquidation, it's collateralValue - remainingDebt expressed in
principal or collateral currency
2 //In case of total liquidation, equity is zero
```

ak

> validators/loan/loan_change_collateral_action.ak (line 209)

```
1 //we subtract the current amount (resulting it as 0) and add the new one.
```

ak

Corrected to:

```
1 //we subtract the current amount (resulting it as 0) and add the new one.
```

ak

> lib/fluidtokens/constants.ak (line 26)

```
1 //7 hours max time heartbeat
```

ak

Corrected to:

```
1 //7 hours max time heartbeat ak
```

> lib/fluidtokens/types/minswap_v2_types.ak (line 99)

```
1 // Withdrawl all amount of the LP token that is available on the order's value ak
```

Corrected to:

```
1 // Withdraw all amount of the LP token that is available on the order's value ak
```

> lib/fluidtokens/types/minswap_v2_types.ak (line 141)

```
1 // SwapExactOut is used for exchanging single asset in the liquidity pool and
  receiving the exact amout of other asset. ak
```

Corrected to:

```
1 // SwapExactOut is used for exchanging single asset in the liquidity pool and
  receiving the exact amount of other asset. ak
```

Recommendation

Review and correct typographical errors and inaccuracies in comments throughout the Fluid Tokens codebase to ensure clarity and prevent misunderstandings.

Resolution

Resolved in commit hash `c51f13c0e89a3eea63d2fee3de788bae7e31a7f9`

[I 113] Configuration NFT Minting Output Conflict

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Output Conflict

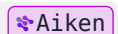
Affected Files:

- `validators/lender-manager/lm_config.ak`
- `validators/config.ak`

Description

In the `lm_config.ak` and `config.ak` minting policies, the output holding the minted NFT is fetched from the first output of the transaction using the line:

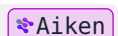
```
1 let output_parameters = utils.safe_list_at(self.outputs, 0)
```

 Aiken

If both the `lm_config_NFT` and `config_NFT` are to be minted in the same transaction, first output cannot satisfy both minting policies, as they both require the NFT to be sent to the first output. This means that only one of the minting policies will be correctly validated, while the other one will fail because it will fetch the wrong output parameters, leading to a failed transaction.

Relevant code snippet (found in both validators):

```
1 mint(_redeemer: Redeemer, policy_id: PolicyId, self: Transaction) {
2     let contractCredential = Script(policy_id)
3
4     ...
5
6     let output_parameters = utils.safe_list_at(self.outputs, 0)
7
8     and {
9         output_parameters.address.payment_credential == contractCredential,
10        ...
11        quantity_of(
12            output_parameters.value,
13            policy_id,
14            constants.config_asset_name,
15        ) == 1,
16    }
17 }
```

 Aiken

Recommendation

Modify `lm_config.ak` to fetch `output_parameters` from the second output instead of the first output:

```
1 let output_parameters = utils.safe_list_at(self.outputs, 1)
```

 Aiken

This change will allow both `lm_config_NFT` and `config_NFT` to be minted in the same transaction without any validation issues, as each minting policy will fetch the correct output parameters from their respective outputs.

Resolution

Client acknowledged the finding and mentioned that since `config.ak` is already live, both NFTs will never be minted together in the same transaction, thus no immediate fix is necessary.

End of Report



No Witness Labs — Confidential

Copyright © 2026